COMPARAR LA IMPLEMENTACIÓN DE UN DICCIONARIO DE DATOS FISICO EN DIFERENTES SGBD

PRESENTADO POR:

-NATHALY RIVAS ARISTIZABAL.

CURSO:
BASES DE DATOS.

TUTORA:
ORALIA CORTÉS CARVAJAL.

G100

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO.
2026

¿POR QUÉ ELEGÍ ESTOS TRES PROGRAMAS?

Para este proyecto elegí tres de los sistemas de bases de datos más utilizados en el mundo real, con el fin de probar que mi diseño de 15 tablas funciona correctamente en cualquier entorno:

PostgreSQL: Lo elegí porque es un programa libre, muy estable y bastante estricto con las reglas de organización de los datos.

MySQL: Lo elegí por ser el más común en el desarrollo de páginas web y aplicaciones, además de ser muy fácil de usar mediante herramientas visuales como MySQL Workbench.

Microsoft SQL Server: Lo elegí porque es el programa preferido por las empresas que trabajan con la tecnología de Microsoft.

EXPLICACIÓN PASO A PASO DE LA IMPLEMENTACIÓN DEL DICCIONARIO DE DATOS FÍSICO

El proceso para construir y verificar la base de datos de la Red Social Estudiantil, compuesta por 15 tablas estudiante, perfil, interés, habilidad, publicación, comentario, reacción, grupo, evento, mentoría, seguimiento y sus cuatro tablas de intersección, lo desarrollé siguiendo estos pasos:

Paso 1: Ajuste y adaptación del código SQL Revisé la sintaxis requerida por cada programa para la generación automática de los identificadores principales claves primarias. En PostgreSQL utilicé el tipo SERIAL, en MySQL usé la propiedad AUTO_INCREMENT, y en SQL Server apliqué IDENTITY(1,1).

Paso 2: Creación ordenada de la estructura Para evitar errores de dependencia, ejecuté la creación de las tablas en un orden lógico estricto:

En primer lugar creé las tablas independientes o catálogos base estudiante, interés, habilidad, grupo.

En segundo lugar creé las tablas que dependen de las anteriores perfil, publicación, comentario, reacción, evento, mentoría, seguimiento.

En tercer lugar creé las tablas de unión para las relaciones de muchos a muchos estudiante_interes, estudiante_habilidad, miembro_grupo, asistente_evento. A todas les asigné sus respectivas reglas de conexión mediante llaves foráneas con borrado en cascada ON DELETE CASCADE cuando correspondió.

Paso 3: Verificación mediante consultas al catálogo del sistema Para confirmar que las 15 tablas y sus columnas se hubieran creado de forma correcta, escribí una consulta de introspección en cada motor para agrupar en una sola línea el nombre de la tabla y sus atributos:

- En PostgreSQL usé la función string_agg() sobre information_schema.columns.
- En MySQL usé la función GROUP_CONCAT() especificando el nombre del esquema.

- En SQL Server usé la función STRING_AGG() ordenada por la posición de las columnas.

DIFICULTADES ENCONTRADAS, VENTAJAS Y DESVENTAJAS DE CADA CASO

PostgreSQL - Probado mediante OneCompiler

Ventajas: Su sintaxis es muy ordenada, exige un estándar de calidad alto al definir reglas y restricciones, y la consulta para agrupar atributos funcionó con mucha rapidez.

Desventajas: Es bastante estricto con el uso de comillas y la diferencia entre mayúsculas y minúsculas en los nombres de las tablas.

Dificultades: Al ejecutarlo en entornos web de prueba, tuve que ajustar la consulta para que no intentara leer esquemas restringidos del sistema que requerían permisos de administrador.

MySQL - Probado en MySQL Workbench

Ventajas: La interfaz gráfica de MySQL Workbench hace que todo el proceso sea intuitivo, permitiendo identificar fácilmente dónde está cualquier falla de sintaxis.

Desventajas: El espacio para mostrar listas de texto agrupadas tiene un límite predeterminado, por lo que si una tabla tiene muchos atributos, puede llegar a recortar la salida.

Dificultades: Durante el proceso de prueba tuve que asegurarme de limpiar correctamente la base de datos previa para que no se generaran bloqueos por llaves foráneas al intentar recrear las tablas.

Microsoft SQL Server - Probado en OneCompiler / VS Code

Ventajas: Es un sistema muy organizado, robusto y el resultado final agrupado por tablas quedó con una presentación muy clara y fácil de leer.

Desventajas: Depende de tener instalado y activo el servicio del servidor en segundo plano en la computadora para poder conectar herramientas externas como VS Code.

Dificultades: Al intentar conectarlo desde la extensión de SQL Server en VS Code, me apareció un error de red Error 40: named pipes provider debido a que mi computadora no contaba con el motor de SQL Server instalado de forma local. Para resolver esta dificultad de manera práctica y sin perder tiempo instalando programas pesados, migré la ejecución al entorno en línea OneCompiler con soporte T-SQL, donde obtuve el resultado esperado de inmediato.

PROPUESTAS DE MEJORA Y OPTIMIZACIÓN DEL PROYECTO

Creación de índices para búsquedas rápidas: Propongo agregar índices secundario en las llaves foráneas que más se van a consultar como el id_estudiante dentro de las tablas de

publicaciones y comentarios para que las búsquedas sean mucho más rápidas cuando la red social tenga miles de usuarios.

Automatización del despliegue: Considero útil implementar herramientas como Flyway para que las actualizaciones o modificaciones a la estructura de las tablas se puedan aplicar automáticamente en cualquier base de datos con un solo comando.

Independencia de listas fijas: Para campos como el tipo de publicación pregunta, recurso, noticia, meme o el estado de asistencia, sería conveniente crear tablas de catálogo separadas. De esta manera, si en el futuro se quiere agregar una nueva categoría, no será necesario modificar la estructura de la tabla principal.

CONCLUSIONES Y REFLEXIÓN FINAL

Conclusión del proceso individual: A lo largo de este ejercicio logré comprender las diferencias prácticas que existen al trabajar con distintos gestores de bases de datos. Lo más valioso de esta experiencia fue desarrollar la capacidad de adaptar el código a cada motor y resolver sobre la marcha los problemas de conexión técnica que se presentaron con SQL Server, buscando alternativas eficientes en línea como OneCompiler para cumplir con los objetivos trazados.

Conclusión del diseño de la base de datos: Comprobé de primera mano que el diseño relacional que elaboré para la Red Social Estudiantil está bien estructurado, cumple con las normas de diseño (3FN) y no presenta información duplicada ni innecesaria. La verificación exitosa de las 15 tablas en PostgreSQL, MySQL y SQL Server me da la tranquilidad de que la base de datos es totalmente funcional.

Video:

https://youtu.be/Nk-JuA5oRk0

PostgreSQL

tabla	atributo	tipo_dato	permite_nulo
asistente_evento	id_evento	integer	NO
asistente_evento	id_estudiante	integer	NO
asistente_evento	estado_asistencia	character varying	YES
comentario	id_comentario	integer	NO
comentario	id_publicacion	integer	NO
comentario	id_estudiante	integer	NO
comentario	contenido	text	NO
comentario	fecha_creacion	timestamp without time zone	YES
estudiante	id_estudiante	integer	NO
estudiante	correo_institucional	character varying	NO
estudiante	pasword_hash	character varying	NO
estudiante	fecha_registro	timestamp without time zone	YES
estudiante_habilidad	id_estudiante	integer	NO
estudiante_habilidad	id_habilidad	integer	NO
estudiante_habilidad	nivel	character varying	YES
estudiante_interes	id_estudiante	integer	NO
estudiante_interes	id_interes	integer	NO
evento	id_evento	integer	NO
evento	id_grupo	integer	YES
evento	titulo	character varying	NO
evento	fecha_inicio	timestamp without time zone	NO
evento	ubicacion_geo	jsonb	YES
grupo	id_grupo	integer	NO
grupo	nombre	character varying	NO
grupo	descripcion	text	YES
grupo	tipo	character varying	YES
habilidad	id_habilidad	integer	NO
habilidad	nombre_habilidad	character varying	NO
habilidad	categoria	character varying	YES
interes	id_interes	integer	NO
interes	nombre_interes	character varying	NO
interes	descripcion	text	YES
mentoria	id_mentoria	integer	NO
mentoria	id_mentor	integer	NO
mentoria	id_estudiante	integer	NO
mentoria	materia_tema	character varying	NO
mentoria	estado	character varying	YES
miembro_grupo	id_grupo	integer	NO
miembro_grupo	id_estudiante	integer	NO
miembro_grupo	rol_grupo	character varying	YES
perfil	id_perfil	integer	NO
perfil	id_estudiante	integer	NO
perfil	nombre_completo	character varying	NO
perfil	carrera	character varying	NO
perfil	semestre	integer	YES
perfil	preferencias_intereses	jsonb	YES
publicacion	id_publicacion	integer	NO
publicacion	id_estudiante	integer	NO
publicacion	contenido	text	NO
publicacion	tipo_publicacion	character varying	YES
publicacion	fecha_creacion	timestamp without time zone	YES
reaccion	id_reaccion	integer	NO
reaccion	id_publicacion	integer	NO
reaccion	id_estudiante	integer	NO
reaccion	tipo_reaccion	character varying	YES
seguimiento	id_seguimiento	integer	NO
seguimiento	id_seguidor	integer	NO
seguimiento	id_seguido	integer	NO
seguimiento	fecha_seguimiento	timestamp without time zone	YES

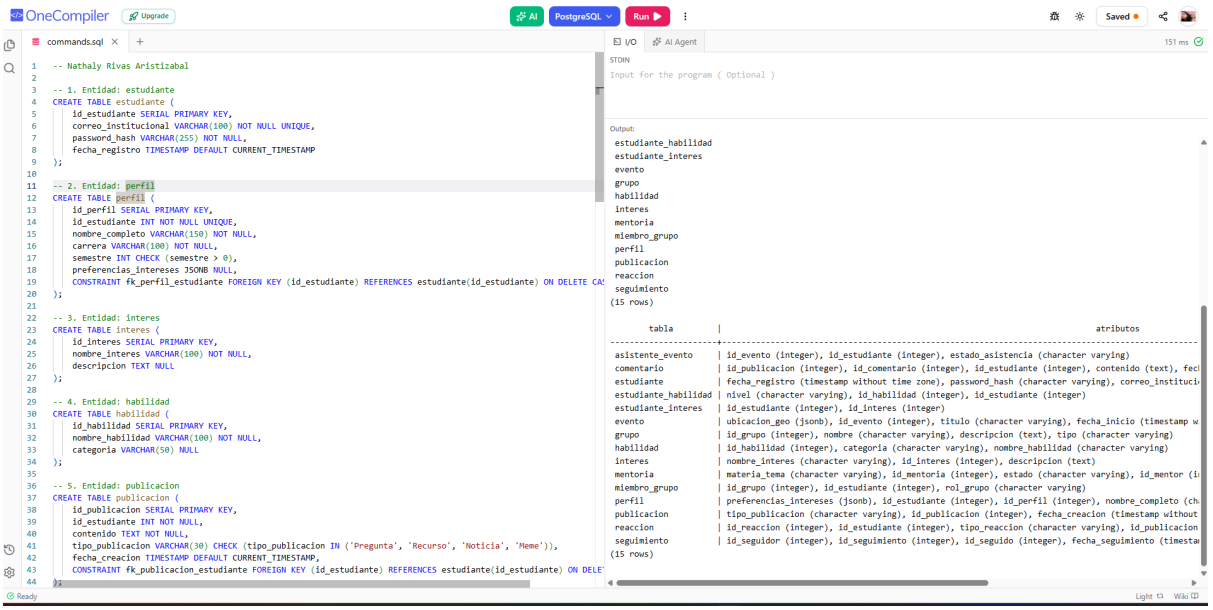
(59 rows)

Output:

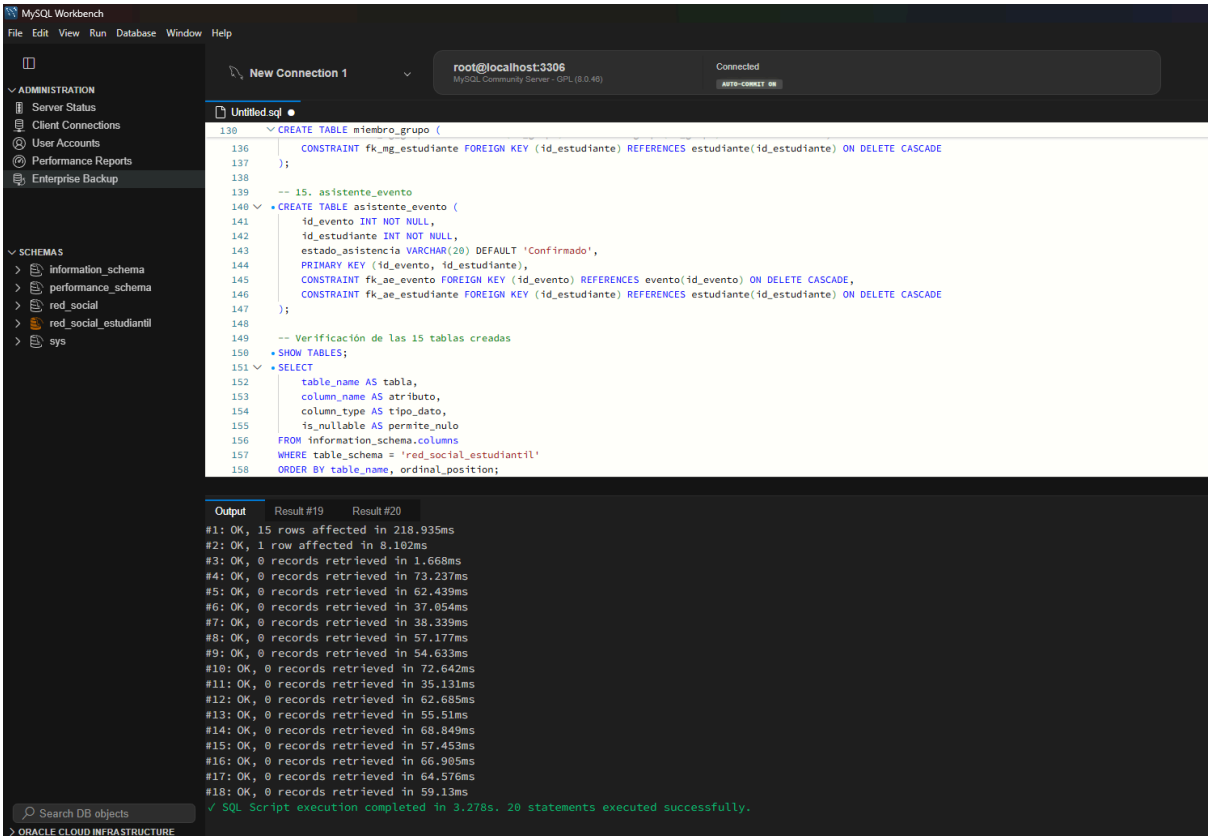
```
 estudiante
 estudiante_habilidad
 estudiante_interes
 evento
 grupo
 habilidad
 interes
 mentoria
 miembro_grupo
 perfil
 publicacion
 reaccion
 seguimiento
(15 rows)
```

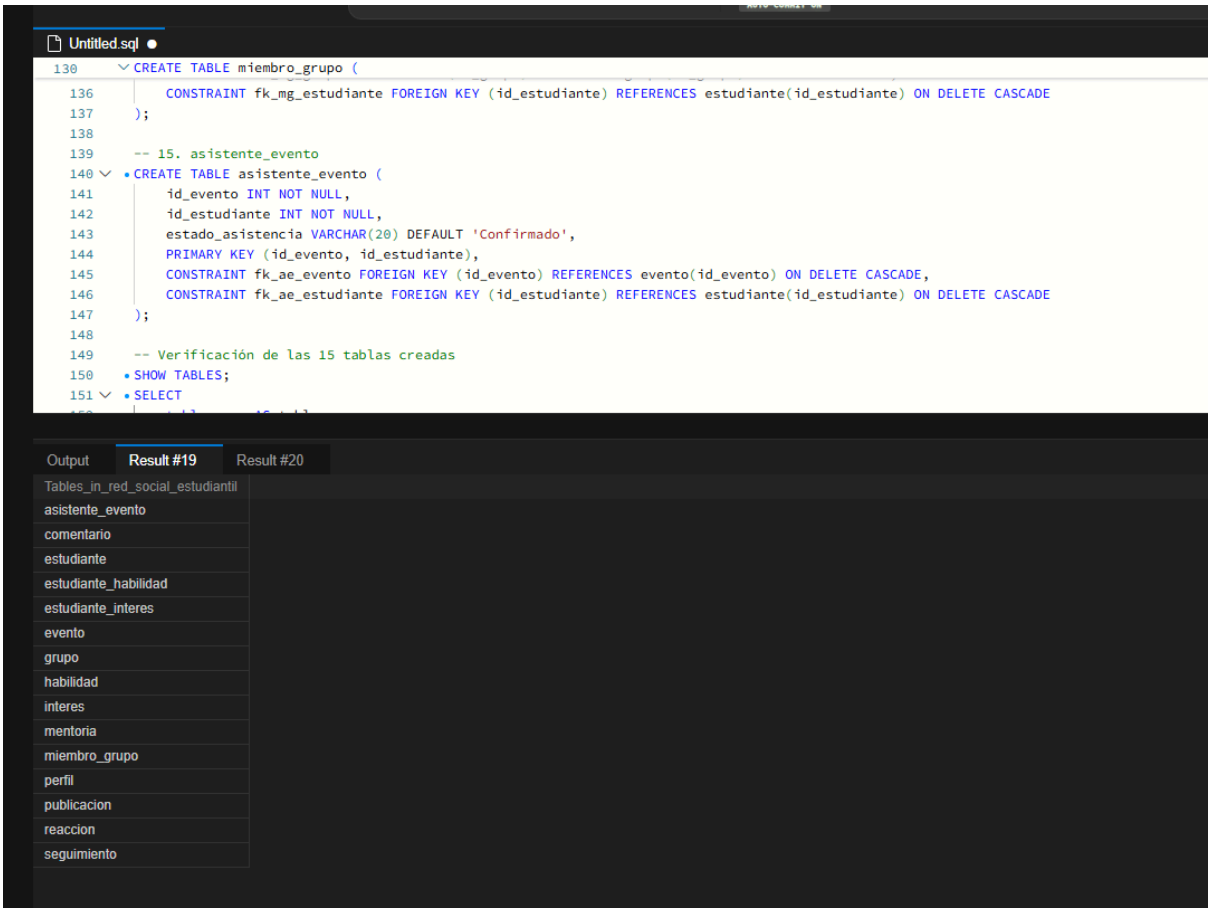
tabla	atributos
asistente_evento	id_evento (integer), id_estudiante (integer), estado_asistencia (character varying)
comentario	id_publicacion (integer), id_comentario (integer), id_estudiante (integer), contenido (text), fecha_creacion (timestamp without time zone)
estudiante	fecha_registro (timestamp without time zone), password_hash (character varying), correo_institucional (character varying), id_estudiante (integer)
estudiante_habilidad	nivel (character varying), id_habilidad (integer), id_estudiante (integer)
estudiante_interes	id_estudiante (integer), id_interes (integer)
evento	ubicacion_geo (jsonb), id_evento (integer), titulo (character varying), fecha_inicio (timestamp without time zone), id_grupo (integer)
grupo	id_grupo (integer), nombre (character varying), descripcion (text), tipo (character varying)
habilidad	id_habilidad (integer), categoria (character varying), nombre_habilidad (character varying)
interes	nombre_interes (character varying), id_interes (integer), descripcion (text)
mentoria	materia_tema (character varying), id_mentoria (integer), estado (character varying), id_mentor (integer), id_estudiante (integer)
miembro_grupo	id_grupo (integer), id_estudiante (integer), rol_grupo (character varying)
perfil	preferencias_intereses (jsonb), id_estudiante (integer), id_perfil (integer), nombre_completo (character varying), semestre (integer), carrera (character varying)
publicacion	tipo_publicacion (character varying), id_publicacion (integer), fecha_creacion (timestamp without time zone), id_estudiante (integer), contenido (text)
reaccion	id_reaccion (integer), id_estudiante (integer), tipo_reaccion (character varying), id_publicacion (integer)
seguimiento	id_seguidor (integer), id_seguimiento (integer), id_seguido (integer), fecha_seguimiento (timestamp without time zone)

(15 rows)



MYSQL


MySQL Workbench
File Edit View Run Database Window Help
New Connection 1
root@localhost:3306
MySQL Community Server - GPL (8.0.46)
Connected
ADMINISTRATION
Server Status
Client Connections
User Accounts
Performance Reports
Enterprise Backup
SCHEMAS
information_schema
performance_schema
red_social
red_social_estudiantil
sys
Untitled.sql
130 CREATE TABLE miembro_grupo (
136 CONSTRAINT fk_mg_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE
137);
139 -- 15. asistente_evento
140 CREATE TABLE asistente_evento (
141 id_evento INT NOT NULL,
142 id_estudiante INT NOT NULL,
143 estado_asistencia VARCHAR(20) DEFAULT 'Confirmado',
144 PRIMARY KEY (id_evento, id_estudiante),
145 CONSTRAINT fk_ae_evento FOREIGN KEY (id_evento) REFERENCES evento(id_evento) ON DELETE CASCADE,
146 CONSTRAINT fk_ae_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE
147);
149 -- Verificación de las 15 tablas creadas
150 SHOW TABLES;
151 SELECT
152 table_name AS tabla,
153 column_name AS atributo,
154 column_type AS tipo_dato,
155 is_nullable AS permite_nulo
156 FROM information_schema.columns
157 WHERE table_schema = 'red_social_estudiantil'
158 ORDER BY table_name, ordinal_position;
Output
Result #19
Result #20
#1: OK, 15 rows affected in 218.935ms
#2: OK, 1 row affected in 8.102ms
#3: OK, 0 records retrieved in 1.668ms
#4: OK, 0 records retrieved in 73.237ms
#5: OK, 0 records retrieved in 62.439ms
#6: OK, 0 records retrieved in 37.054ms
#7: OK, 0 records retrieved in 38.339ms
#8: OK, 0 records retrieved in 57.177ms
#9: OK, 0 records retrieved in 54.633ms
#10: OK, 0 records retrieved in 72.642ms
#11: OK, 0 records retrieved in 35.131ms
#12: OK, 0 records retrieved in 62.685ms
#13: OK, 0 records retrieved in 55.51ms
#14: OK, 0 records retrieved in 68.849ms
#15: OK, 0 records retrieved in 57.453ms
#16: OK, 0 records retrieved in 66.905ms
#17: OK, 0 records retrieved in 64.576ms
#18: OK, 0 records retrieved in 59.13ms
✓ SQL Script execution completed in 3.278s. 20 statements executed successfully.
Search DB objects
ORACLE CLOUD INFRASTRUCTURE


Untitled.sql
130 CREATE TABLE miembro_grupo (
136 CONSTRAINT fk_mg_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE
137);
139 -- 15. asistente_evento
140 CREATE TABLE asistente_evento (
141 id_evento INT NOT NULL,
142 id_estudiante INT NOT NULL,
143 estado_asistencia VARCHAR(20) DEFAULT 'Confirmado',
144 PRIMARY KEY (id_evento, id_estudiante),
145 CONSTRAINT fk_ae_evento FOREIGN KEY (id_evento) REFERENCES evento(id_evento) ON DELETE CASCADE,
146 CONSTRAINT fk_ae_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE
147);
149 -- Verificación de las 15 tablas creadas
150 SHOW TABLES;
151 SELECT
Output
Result #19
Result #20
Tables_in_red_social_estudiantil
asistente_evento
comentario
estudiante
estudiante_habilidad
estudiante_interes
evento
grupo
habilidad
interes
mentoria
miembro_grupo
perfil
publicacion
reaccion
seguimiento

Output | Result #19 | **Result #20**

tabla	atributo	tipo_dato	permite_nulo
asistente_evento	id_evento	int	NO
asistente_evento	id_estudiante	int	NO
asistente_evento	estado_asistencia	varchar(20)	YES
comentario	id_comentario	int	NO
comentario	id_publicacion	int	NO
comentario	id_estudiante	int	NO
comentario	contenido	text	NO
comentario	fecha_creacion	timestamp	YES
estudiante	id_estudiante	int	NO
estudiante	correo_institucional	varchar(100)	NO
estudiante	password_hash	varchar(255)	NO
estudiante	fecha_registro	timestamp	YES
estudiante_habilidad	id_estudiante	int	NO
estudiante_habilidad	id_habilidad	int	NO
estudiante_habilidad	nivel	varchar(30)	YES
estudiante_interes	id_estudiante	int	NO
estudiante_interes	id_interes	int	NO
evento	id_evento	int	NO
evento	id_grupo	int	YES
evento	titulo	varchar(150)	NO
evento	fecha_inicio	datetime	NO
evento	ubicacion_geo	json	YES
grupo	id_grupo	int	NO
grupo	nombre	varchar(100)	NO
grupo	descripcion	text	YES
grupo	tipo	varchar(30)	YES
interes	id_interes	int	NO
interes	nombre_interes	varchar(100)	NO
interes	descripcion	text	YES
mentoria	id_mentoria	int	NO
mentoria	id_mentor	int	NO
mentoria	id_estudiante	int	NO
mentoria	materia_tema	varchar(100)	NO
mentoria	estado	varchar(20)	YES
miembro_grupo	id_grupo	int	NO
miembro_grupo	id_estudiante	int	NO
miembro_grupo	rol_grupo	varchar(30)	YES
perfil	id_perfil	int	NO
perfil	id_estudiante	int	NO
perfil	nombre_completo	varchar(150)	NO
perfil	carrera	varchar(100)	NO
perfil	semestre	int	YES
perfil	preferencias_intereses	json	YES
publicacion	id_publicacion	int	NO
publicacion	id_estudiante	int	NO
publicacion	contenido	text	NO
publicacion	tipo_publicacion	varchar(30)	YES
publicacion	fecha_creacion	timestamp	YES

reaccion	id_reaccion	int	NO
reaccion	id_publicacion	int	NO
reaccion	id_estudiante	int	NO
reaccion	tipo_reaccion	varchar(20)	YES
seguimiento	id_seguimiento	int	NO
seguimiento	id_seguidor	int	NO
seguimiento	id_seguido	int	NO
seguimiento	fecha_seguimiento	timestamp	YES

SQL Server

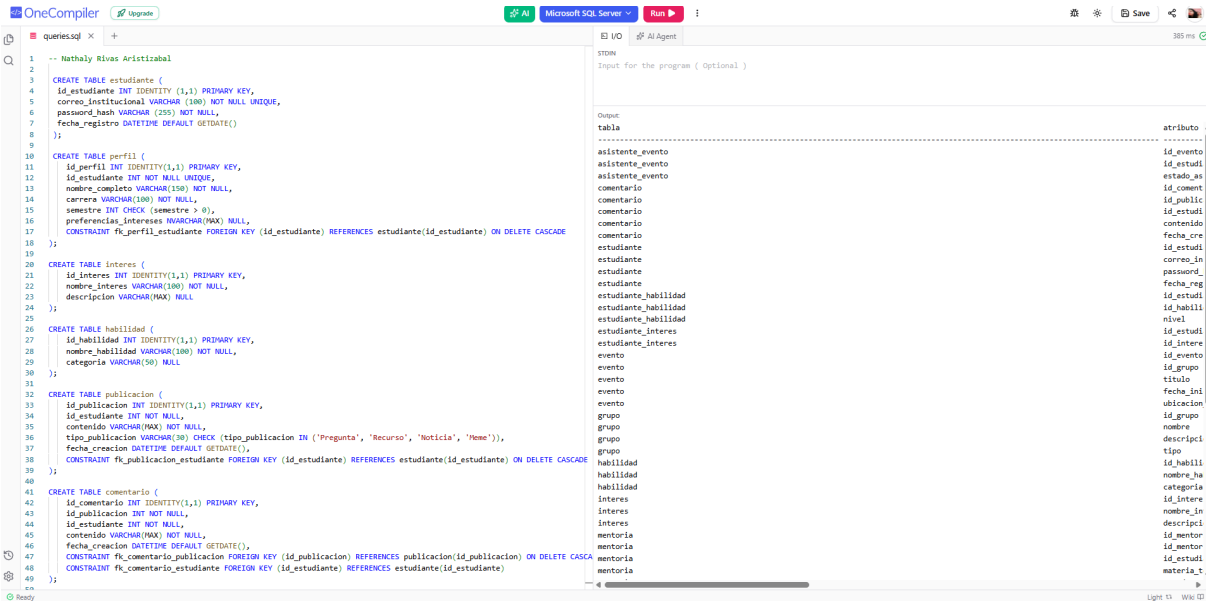
Output:

tabla	atributos
asistente_evento	id_evento (int), id_estudiante (int), estado_asiste
comentario	id_comentario (int), id_publicacion (int), id_estud
estudiante	id_estudiante (int), correo_institucional (varchar)
estudiante_habilidad	id_estudiante (int), id_habilidad (int), nivel (var
estudiante_interes	id_estudiante (int), id_interes (int)
evento	id_evento (int), id_grupo (int), titulo (varchar),
grupo	id_grupo (int), nombre (varchar), descripcion (varc
habilidad	id_habilidad (int), nombre_habilidad (varchar), cat
interes	id_interes (int), nombre_interes (varchar), descrip
mentoria	id_mentoria (int), id_mentor (int), id_estudiante (
miembro_grupo	id_grupo (int), id_estudiante (int), rol_grupo (var
perfil	id_perfil (int), id_estudiante (int), nombre_comple
publicacion	id_publicacion (int), id_estudiante (int), contenic
reaccion	id_reaccion (int), id_publicacion (int), id_estudia
seguimiento	id_seguimiento (int), id_seguidor (int), id_seguido

	atributos
	id_evento (int), id_estudiante (int), estado_asistencia (varchar)
	id_comentario (int), id_publicacion (int), id_estudiante (int), contenido (varchar), fecha_creacion (datetime)
	id_estudiante (int), correo_institucional (varchar), password_hash (varchar), fecha_registro (datetime)
	id_estudiante (int), id_habilidad (int), nivel (varchar)
	id_estudiante (int), id_interes (int)
	id_evento (int), id_grupo (int), titulo (varchar), fecha_inicio (datetime), ubicacion_geo (nvarchar)
	id_grupo (int), nombre (varchar), descripcion (varchar), tipo (varchar)
	id_habilidad (int), nombre_habilidad (varchar), categoria (varchar)
	id_interes (int), nombre_interes (varchar), descripcion (varchar)
	id_mentoria (int), id_mentor (int), id_estudiante (int), materia_tema (varchar), estado (varchar)
	id_grupo (int), id_estudiante (int), rol_grupo (varchar)
	id_perfil (int), id_estudiante (int), nombre_completo (varchar), carrera (varchar), semestre (int), preferencias_intereses (nvarchar)
	id_publicacion (int), id_estudiante (int), contenido (varchar), tipo_publicacion (varchar), fecha_creacion (datetime)
	id_reaccion (int), id_publicacion (int), id_estudiante (int), tipo_reaccion (varchar)
	id_seguimiento (int), id_seguidor (int), id_seguido (int), fecha_seguimiento (datetime)

Output:

tabla	atributo	tipo_dato
asistente_evento	id_evento	int
asistente_evento	id_estudiante	int
asistente_evento	estado_asistencia	varchar
comentario	id_comentario	int
comentario	id_publicacion	int
comentario	id_estudiante	int
comentario	contenido	varchar
comentario	fecha_creacion	datetime
estudiante	id_estudiante	int
estudiante	correo_institucional	varchar
estudiante	password_hash	varchar
estudiante	fecha_registro	datetime
estudiante_habilidad	id_estudiante	int
estudiante_habilidad	id_habilidad	int
estudiante_habilidad	nivel	varchar
estudiante_interes	id_estudiante	int
estudiante_interes	id_interes	int
evento	id_evento	int
evento	id_grupo	int
evento	titulo	varchar
evento	fecha_inicio	datetime
evento	ubicacion_geo	nvarchar
grupo	id_grupo	int
grupo	nombre	varchar
grupo	descripcion	varchar
grupo	tipo	varchar
habilidad	id_habilidad	int
habilidad	nombre_habilidad	varchar
habilidad	categoria	varchar
interes	id_interes	int
interes	nombre_interes	varchar
interes	descripcion	varchar
mentoria	id_mentoria	int
mentoria	id_mentor	int
mentoria	id_estudiante	int
mentoria	materia_tema	varchar
mentoria	estado	varchar
miembro_grupo	id_grupo	int
miembro_grupo	id_estudiante	int
miembro_grupo	rol_grupo	varchar
perfil	id_perfil	int
perfil	id_estudiante	int
perfil	nombre_completo	varchar
perfil	carrera	varchar
perfil	semestre	int
perfil	preferencias_intereses	nvarchar
publicacion	id_publicacion	int
publicacion	id_estudiante	int
publicacion	contenido	varchar
publicacion	tipo_publicacion	varchar
publicacion	fecha_creacion	datetime
reaccion	id_reaccion	int
reaccion	id_publicacion	int
reaccion	id_estudiante	int
reaccion	tipo_reaccion	varchar
seguimiento	id_seguimiento	int
seguimiento	id_seguidor	int
seguimiento	id_seguido	int
seguimiento	fecha_seguimiento	datetime




```sql
-- Nathaly Rivas Aristizabal

CREATE TABLE estudiante (
 id_estudiante INT IDENTITY (1,1) PRIMARY KEY,
 correo_institucional VARCHAR (100) NOT NULL UNIQUE,
 password_hash VARCHAR (255) NOT NULL,
 fecha_registro DATETIME DEFAULT GETDATE()
 );

CREATE TABLE perfil (
    id_perfil INT IDENTITY(1,1) PRIMARY KEY,
    id_estudiante INT NOT NULL UNIQUE,
    nombre_completo VARCHAR(150) NOT NULL,
    carrera VARCHAR(100) NOT NULL,
    semestre INT CHECK (semestre > 0),
    preferencias_intereses NVARCHAR(MAX) NULL,
    CONSTRAINT fk_perfil_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE
);

CREATE TABLE interes (
    id_interes INT IDENTITY(1,1) PRIMARY KEY,
    nombre_interes VARCHAR(100) NOT NULL,
    descripcion VARCHAR(MAX) NULL
);

CREATE TABLE habilidad (
    id_habilidad INT IDENTITY(1,1) PRIMARY KEY,
    nombre_habilidad VARCHAR(100) NOT NULL,
    categoria VARCHAR(50) NULL
);

CREATE TABLE publicacion (
    id_publicacion INT IDENTITY(1,1) PRIMARY KEY,
    id_estudiante INT NOT NULL,
    contenido VARCHAR(MAX) NOT NULL,
    tipo_publicacion VARCHAR(30) CHECK (tipo_publicacion IN ('Pregunta', 'Recurso', 'Noticia', 'Meme')),
    fecha_creacion DATETIME DEFAULT GETDATE(),
    CONSTRAINT fk_publicacion_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante) ON DELETE CASCADE
);

CREATE TABLE comentario (
    id_comentario INT IDENTITY(1,1) PRIMARY KEY,
    id_publicacion INT NOT NULL,
    id_estudiante INT NOT NULL,
    contenido VARCHAR(MAX) NOT NULL,
    fecha_creacion DATETIME DEFAULT GETDATE(),
    CONSTRAINT fk_comentario_publicacion FOREIGN KEY (id_publicacion) REFERENCES publicacion(id_publicacion) ON DELETE CASCA
    CONSTRAINT fk_comentario_estudiante FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante)
);
```